数据结构课程实践报告

城市交通路径导航系统

设 计 与 实 现

课程名称： 数据结构

实践项目： 城市交通路径导航系统

选题方向： 选题 A　图与路径规划

组　　号： _____

姓　　名： _____

学　　号： _____

班　　级： _____

指导教师： _____

2026 年 10 月

项目分工

本项目按「数据层 → 算法层 → 界面层 → 渲染与工程化」四个阶段推进，每个阶段有明确的交付文件与验收口径，分工如表 0-1 所示。

项目分工一览

阶段	负责内容	主要文件	姓名
P1 数据层 (core/loader.py, core/graph.py, data/)	解析 JSON / TXT 两种数据格式、统一字段校验；把线路建成「站台节点 + 邻接表」的带权无向图，计算距离与时间权重	core/loader.py core/graph.py data/*.json|txt	
P2 算法层 (core/algorithms.py)	实现二叉堆 Dijkstra、可采纳启发函数的 A*、以换乘次数为层数的分层 BFS，统一封装 RouteResult，并给出复杂度与适用场景分析	core/algorithms.py	
P3 界面层 (ui/window.py)	搭建 Tkinter 窗口与控件，实现起点/终点选择、算法与速度切换、统计面板、暂停/跳过/复位/导出等按钮的状态机	ui/window.py	
P4 渲染与工程化 (ui/canvas.py, selfcheck.py, tools/, 报告统稿)	画布渲染与交互（等比投影、站名避让、路径高亮、动画时间轴、导出）；10 项自检矩阵与合并闸门；对比实验、报告生成与统稿检查	ui/canvas.py selfcheck.py tools/ 报告终稿	

说明：姓名一列由组内填写；P1~P4 的交付物与验收证据分别见 docs/ 目录下的验收说明与报告第 4 章。

摘　要

城市轨道交通网络规模大、换乘关系复杂，乘客出行往往需要同时权衡“用时最短”与“换乘最少”两类需求。本实践以北京地铁为原型，构建了包含 134 个站点、7 条线路、154 个站台节点的教学简化路网，把线路与换乘关系抽象为带权无向图，设计并实现了一个具有图形界面的城市交通路径导航系统。系统采用分层设计：数据层负责 JSON/TXT 解析与字段校验；图结构层用“站台节点（站点, 线路）+ 邻接表”精确建模换乘；算法层用二叉堆 Dijkstra 求最快路径、用可采纳的直线距离启发函数实现 A*、用分层 BFS 求最少换乘；界面层基于 Tkinter，由 CanvasView 完成等比投影、缩放拖拽、站名避让、路径高亮、逐帧动画与导出。

实验在 134 站路网上选取 5 对起终点、对 3 种算法各计时 5 次并取中位数：A* 与 Dijkstra 的最短时间完全一致，平均扩展节点数由 133.0 降为 76.8，减少 42.3%；BFS 在“慈寿寺→崇文门”上用 1 次换乘换取了比 Dijkstra 多 2.6 分钟的行程，直观体现了“最少换乘 ≠ 最快”的准则差异。系统内置 10 项自检矩阵（数据、算法、边界、性能、界面）并全部通过，退出码为 0；动画与立即显示共用同一份 PathView，统计值逐键相等；缩放 2.5 倍后点击命中误差 11.0 像素（≤12 像素）；1400×900 画布首绘约 200 ms，导出 PNG 约 3 s。结果表明，三种算法分别适用于“最快到达”“快速启发式搜索”“少换乘出行”等不同场景，分层设计也让算法、渲染与自检可以独立演进。

关键词：图论；站台节点图；邻接表；Dijkstra；A*；BFS；Tkinter

目　录

第 1 章　需求分析与问题建模 6
1.1　项目背景 6
1.2　功能需求 6
1.3　图模型 G=(V,E) 7
1.4　换乘建模与权重设计 8
1.5　数据来源与规模 8
第 2 章　数据结构设计 10
2.1　总体架构 10
2.2　邻接表的选择 11
2.3　核心数据结构 12
2.4　优先队列在 Dijkstra 中的应用 12
2.5　数据文件格式与解析 13
第 3 章　算法设计与实现 14
3.1　Dijkstra 最短耗时算法 14
3.2　A* 启发式搜索 15
3.3　BFS 最少换乘算法 15
3.4　复杂度与适用场景对比 16
3.5　异常处理 16
3.6　系统实现与界面概览 17
第 4 章　正确性验证与结果分析 19
4.1　运行环境与启动方式 19
4.2　界面与功能演示 19
4.3　渲染与交互实现 20
4.4　异常与边界测试 23
4.5　正确性验证 24
4.6　对比实验 25
4.7　性能与观感 27
第 5 章　总结与收获 28

5.1 图、堆、队列在本项目中的实际作用 28
5.2 三算法在效率、公平与可解释性上的差异 28
5.3 开发中的问题与解法 29
5.4 局限与改进 30
参考文献 31
附录 A 复现命令 32
附录 B 文件清单 33
附录 C 自检输出节选 34

第 1 章　需求分析与问题建模

本章要解决的是“把现实出行问题翻译成图论问题”：先说明为什么地铁导航适合用图来建模，再把系统需要提供的能力整理成可验收的功能需求，最后给出图模型、换乘建模方式与数据规模，为第 2 章的数据结构选型提供依据。

1.1　项目背景

城市交通网络中的路径导航是图论算法的典型应用。地铁线路可以看作由站点和区间构成的网络，乘客从起点到终点的一次出行，本质上是在这张网络上寻找一条满足某种优化目标的路径。与一般最短路问题不同的是，地铁出行还必须考虑换乘：换乘次数过多会显著降低出行体验，而单纯追求换乘最少又可能绕行过远、总耗时变长。因此，一个实用的导航系统需要同时支持“最快路径”“最少换乘”等不同搜索准则，并让用户直观地看到线路、换乘点与最终路径。

本实践的任务是：把现实地铁网络抽象为图，设计合适的数据结构进行存储，实现 Dijkstra、A*、BFS 三种算法，并开发一个具备图形界面的导航模拟系统。项目强调从数据建模、数据结构选择、算法实现到界面交互的完整软件工程流程，同时用统一的实验数据比较不同算法的效率与结果差异，并用自动化自检矩阵保证“改坏实现会变红”，而不是只靠肉眼观察。

1.2　功能需求

根据任务书要求，系统需要覆盖地图加载与存储、多准则搜索、交互式界面、异常处理、自检与算法对比等功能。经过需求整理，功能需求如表 1-1 所示。

表 1-1　系统功能需求

功能模块	具体要求	实现位置
地图加载	从 .json 或 .txt 文件读取站点坐标与线路连接关系，非法数据给出中文提示	core/loader.py：load_dataset + 字段校验
网络可视化	绘制线路、站点、换乘站、站名与图例，任意窗口尺寸不变形	ui/canvas.py：ViewTransform + 线路/站点绘制
起终点选择	下拉框选择，或在地图上左键/右键点选；缩放后仍能命中	ui/window.py + CanvasView.bind_click / hit_test
最快路径	以乘车时间 + 换乘时间为边权运行 Dijkstra	core/algorithms.py：二叉堆实现
最少换乘	以换乘次数为层数的 BFS，给出换乘站，并在同换乘次数下择优	bfs_min_transfers + 约束最短路
启发式搜索	以直线距离换算时间为 h(n) 的 A*，结果仍最优	astar：f(n) = g(n) + h(n)

功能模块	具体要求	实现位置
结果显示	路径高亮、分段 km/min 标注、途经站点清单、耗时/里程/换乘/扩展节点统计	ui/canvas.py + 统计面板
动画演示	逐帧回放搜索与路径过程，支持 1×/2×/4× 变速、暂停、跳过	CanvasView.animate / toggle_pause / skip_animation
导出	导出 PNG（截图）或 PS（矢量）用于报告与打印	CanvasView.export + tools/export_canvas.py
自检与对比	10 项自动断言 + 5 对起终点 × 3 算法对比实验	selfcheck.py、main.py --check、tools/bench.py

表 1-1 中的每一项都在最终仓库里有对应实现与验证手段：加载与校验对应自检第 01、02 项，三种算法对应第 03、04、05 项，边界对应第 06、07 项，性能对应第 08 项，渲染与交互对应第 09、10 项，因此需求不是“写在纸上的愿望”，而是可被自动检查的契约。

1.3 图模型 G=(V,E)

把地铁网络抽象为带权无向图 G=(V,E)。物理意义下，V 为所有站点，E 为相邻站点之间的轨道区间；为了精确描述换乘，搜索时进一步把每个“站点—线路”组合定义为一个站台节点，并在同一物理站点的不同站台之间加入换乘边。这样，乘车与换乘两类行为都可以统一表示为图中的边，Dijkstra、A*、BFS 只需面对同一张图。

顶点与边的属性定义如表 1-2 所示。站点与线路对象都是不可变数据（frozen dataclass），建图阶段一次性生成站台节点、乘车边与换乘边，搜索阶段只读，避免算法运行过程中修改图结构导致结果不可复现。

表 1-2　顶点与边的属性定义

对象	属性	含义	示例
站点顶点	id	站点唯一编号	S012
	name	站名	复兴门
	x, y	示意坐标（km，y 轴朝上）	(-3.6, -0.12)
线路	id / name	线路编号与名称	L1 / 1 号线
	color / loop	绘制颜色 / 是否环线	#C0392B / False
乘车边	to_station, to_line	同线路相邻两站	西单 → 天安门西
	time, distance	乘车时间（min）与里程（km）	2.31 / 1.0
换乘边	站点内两条线路之间	同一站点换到另一条线路	西单：1 号线 → 4 号线
	time, distance	换乘时间 / 里程（里程恒为 0）	4.0 / 0.0

1.4 换乘建模与权重设计

若只把物理站点作为顶点，换乘次数就必须额外记录，算法状态会变得不清晰；而采用“站台节点”后，换乘被显式表示为一条边，路径中每经过一条换乘边，换乘次数就加一。起点站的所有站台初始距离均为 0（多源起点），终点站任一属于终点的站台都可作为目标，因此不需要虚拟超级源点或汇点，也不需要为终点单独处理换乘。

乘车边权重按列车平均旅行速度与停站时间计算：$t = d / v \times 60 + t_dwell$，其中 d 为两站间的欧几里得距离（km），$v = 35$ km/h，$t_dwell = 0.6$ min；换乘边权重取 4.0 min，用于近似站内步行与候车时间。路径总耗时等于所有乘车段与换乘段时间之和，总里程只累加乘车段距离。模型假设列车匀速、各线路速度相同、换乘时间固定，不模拟实时延误与车厢拥挤程度。

这样建模还有一个好处：A* 的启发函数可以直接复用同一份坐标与速度——$h(n) = 直线距离(n, 终点) / v \times 60$，与边权使用同一套几何，因此可采纳性分析是自洽的（详见 3.2 节）。

1.5 数据来源与规模

示例数据以北京地铁为原型手工简化，包含 1、2、4、5、6、10 号线与一条 S1 支线，其中 2 号线、10 号线为环线，共 20 个换乘站。为演示“两站不连通”的异常处理，S1 支线在简化数据中被设计为与主网不连通的孤立分量。数据规模如表 1-3 所示，各线路站点数如表 1-4 所示。

表 1-3　示例路网数据规模（由 core.loader 实测）

指标	数值	指标	数值
物理站点数	134	线路数	7
站台节点数	154	有向边数	338
乘车边（有向）	298	换乘边（有向）	40
相邻区间（无向）	149	换乘站数	20
环线数	2	列车速度	35 km/h
停站时间	0.6 min	换乘时间	4.0 min

表 1-4　各线路站点数

线路	站点数	环线	备注
1 号线	23	否	—

线路	站点数	环线	备注
2 号线	18	是	—
4 号线	24	否	—
5 号线	22	否	—
6 号线	20	否	—
10 号线	44	是	—
S1 支线	3	否	与主网不连通（教学用例）

说明：坐标为教学示意坐标（单位 km），不是真实地理投影；A* 的启发函数与乘车边使用同一份坐标，因此可采纳性分析仍然成立。S1 支线仅用于验证“无路径”边界。

第 2 章　数据结构设计

本章要解决的是“用什么结构存图、用什么结构跑搜索”：先给出系统分层与模块职责，再说明为什么选邻接表而不是邻接矩阵，然后列出核心数据结构、优先队列的使用方式，最后给出两种数据文件的格式与解析规则。

2.1　总体架构

系统按照“高内聚、低耦合”的原则划分为数据加载、图结构、算法引擎与界面四层，外加一层工程化工具（自检、导出、对比实验、报告生成）。各模块职责如表 2-1 所示。算法层只依赖图结构暴露的邻接表、线路顺序与权重计算接口，不依赖任何 Tkinter 代码；界面层只负责调用算法并展示结果。这样既方便写自检，也便于后续替换命令行界面或 Web 前端。

表 2-1　模块划分与职责

文件	主要类/函数	职责
core/graph.py	Station、MetroLine、Edge、MetroGraph	站点/线路对象、站台节点建图、距离与时间权重
core/loader.py	MetroDataset、load_dataset、DataFormatError	解析 JSON/TXT，统一校验并构造 MetroGraph
core/algorithms.py	RouteResult、Leg、dijkstra、astar、bfs_min_transfers	三种寻路算法与结果封装
ui/window.py	MetroNavWindow	Tkinter 窗口、控件、状态机、统计面板（P3）
ui/canvas.py	CanvasView、ViewTransform、PathView	画布渲染、交互、路径高亮、动画、导出（P4）
selfcheck.py	10 项检查函数 + mutation 演示	合并闸门：数据/算法/边界/性能/界面断言
main.py	main、--check、--smoke	程序入口、自检入口与界面冒烟
tools/	export_canvas、bench、record_demo、make_report	导出插图、对比实验、录屏、报告生成与检查

系统中的数据流为：数据文件经加载器解析、校验后构造图对象；窗口把起点、终点和搜索准则交给算法引擎，算法返回统一的 RouteResult；窗口再调用 CanvasView 的公开方法，由画布负责把路径、站点清单和统计数据画出来。模块划分与数据流如图 2-1 所示。

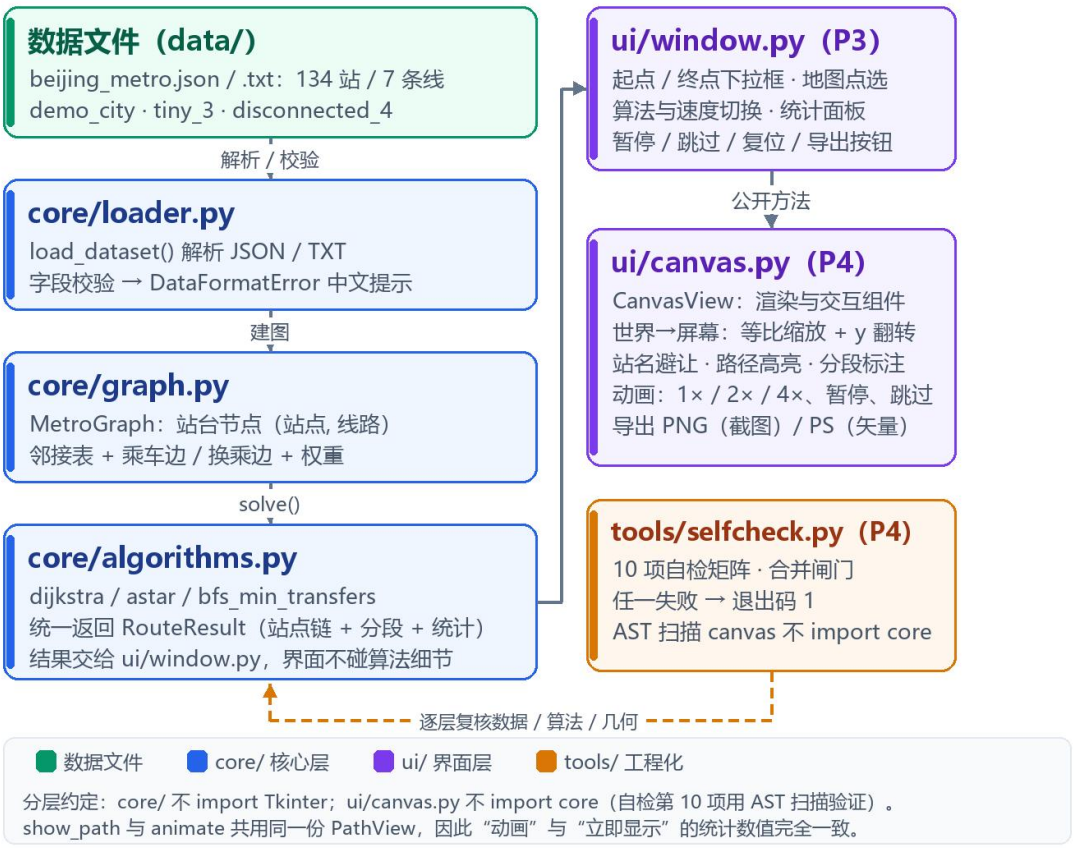


图 2-1　系统模块划分与数据流

图 2-1 中有两条约束特别重要：其一，core/ 不 import Tkinter，算法与界面完全解耦，因此自检可以不开窗口直接验证算法；其二，ui/canvas.py 不 import core，也不读文件，它通过 duck typing 的归一化函数接受任何“像图/像结果”的对象，自检第 10 项会用 AST 扫描源码把这条约束固定下来。

2.2　邻接表的选择

城市轨道交通图是典型的稀疏图：每个站台节点通常只与同线路的前后两站相连，再加少量换乘边，边数与顶点数处于同一数量级。当前数据中站台节点 $|V| = 154$，有向边 $|E| = 338$，平均每个站台节点的出度约为 2.19。若使用邻接矩阵，需要 $154 \times 154 \approx 2.37$ 万个存储单元，且遍历一个站台节点的邻居要扫描整行 O(V)；而邻接表只需保存实际存在的边，空间 O(V+E)，遍历邻居的代价与度数成正比。两种存储方式的对比如表 2-2 所示，因此系统选择“字典 + 列表”的邻接表实现。

表 2-2　邻接表与邻接矩阵的对比

比较项	邻接矩阵	邻接表（本系统）
空间复杂度	$O(V^2)$，稀疏图浪费大	O(V+E)，只存实际边
判断/遍历邻居	O(V)	O(deg(v))
增加/删除边	O(1)	O(1) 追加 / O(deg) 删除
适用场景	稠密图	路网等稀疏图
本数据实际存储	约 23716 个单元	338 条有向边

邻接表使用字典 adjacency 存储：键为（站点 ID, 线路 ID）站台节点，值为 Edge 列表。每条乘车边在两个方向各存一次，因此是无向图的对称表示；换乘边同样双向添加（20 个换乘站、每个站两条线，共 40 条有向换乘边）。为了重建路径时补齐中间站，MetroGraph 还保存了每条线路的站点顺序 line_stations 与“站点→下标”的映射，环线用模运算取站数较少的方向。

2.3　核心数据结构

系统的主要数据结构如表 2-3 所示。它们的分工可以用一句话概括：Station 与 MetroLine 描述输入，Edge 描述图的边，adjacency 系列字典描述图的存储，heap + dist/prev 描述搜索状态，RouteResult 描述输出。

表 2-3　主要数据结构

结构	实现	用途
Station	frozen dataclass：id、name、x、y	物理站点及示意坐标
MetroLine	frozen dataclass：id、name、color、loop、stations	线路及列车站序
Edge	frozen dataclass：to_station、to_line、kind、line_id、time、distance	一条有向边，区分 ride/transfer
adjacency	dict[(station, line)] → list[Edge]	邻接表
station_lines	dict[station] → list[line]	站点所属线路、换乘判断
line_stations	dict[line] → list[station]	环线方向与中间站补齐
heap + dist/prev	heapq + dict	Dijkstra/A* 的优先队列与前驱
visited_order	list[station_id]	供搜索动画使用的扩展顺序
RouteResult	dataclass：stations、legs、total_time、total_distance、transfers、expanded	统一的结果封装，界面只认它

2.4　优先队列在 Dijkstra 中的应用

Dijkstra 每轮需要从“尚未确定最短距离”的站台节点中取出当前距离最小者。若用线性数组扫描，每轮 O(V)、总计 $O(V^2)$；使用二叉堆（Python 的 heapq 小顶堆）后，取最小值 O(log V)、松弛成功时入堆 O(log V)，总复杂度降为 O((V+E) log V)。

系统为每个堆元素附加一个自增计数器 counter，避免距离相同时继续比较站台节点元组导致不可比较或顺序不稳定，同时保证弹出顺序稳定、实验可复现。

A* 使用同一套堆结构，只是排序键由 g(n) 改为 f(n) = g(n) + h(n)。为避免重复处理，弹出时检查 visited 集合；若站台节点已被扩展则跳过。前驱数组 prev 记录“从哪个站台节点、经过哪类边到达”，用于最终回推站台链并生成乘车/换乘分段。

2.5 数据文件格式与解析

系统同时支持 JSON 与 TXT 两种格式。JSON 使用 stations 数组与 lines 数组，适合程序生成和结构化读取；TXT 每行一条记录，适合手工编辑与课堂演示。两种格式的示例分别如下面的代码块所示。

代码 2-1 JSON 数据格式（节选）

```
{
  "meta": {"city": "演示城市",
           "params": {"train_speed_kmh": 35.0, "dwell_minutes": 0.6,
                      "transfer_minutes": 4.0}},
  "stations": [{"id": "S1", "name": "起点站", "x": 0.0, "y": 0.0}],
  "lines": [{"id": "L1", "name": "1 号线", "color": "#E11D48",
             "loop": false, "stations": ["S1", "S2", "S3"]}]
}
```

代码 2-2 TXT 数据格式（节选）

```
CITY 演示城市
PARAM train_speed_kmh=35.0
PARAM dwell_minutes=0.6
PARAM transfer_minutes=4.0
STATION,S001,苹果园,-13.7,0.2
LINE,L1,1 号线,#C0392B,0,S001,S002,S003,...
```

加载器在读取后统一执行校验：站点编号与站名唯一且非空、坐标可转浮点数、线路引用的站点必须存在、同一条线路内站点不重复、线路至少 2 站、环线至少 3 站、速度必须大于 0、停站与换乘时间不能为负。任何错误都会抛出 DataFormatError，并尽可能带上行号或字段名；界面捕获后以中文对话框提示，而不是让程序崩溃。

第 3 章　算法设计与实现

本章要解决的是“三种准则各自怎么算、复杂度如何、出错怎么办、系统如何落地”：依次说明 Dijkstra、A* 与分层 BFS 的实现细节，给出复杂度与适用场景对比，整理异常处理策略，最后从分层视角概览系统实现与主界面。

3.1　Dijkstra 最短耗时算法

Dijkstra 维护源点到各站台节点的最短距离估计 dist，并用优先队列按距离从小到大取出站台节点。取出后对它的每条出边执行松弛（relaxation）：若经过当前站台节点到达邻居更短，则更新邻居的 dist 与前驱，并将邻居入堆。由于所有边权均为正，某个站台节点第一次被弹出时，其 dist 已经是最终最短距离。伪代码如算法 3-1 所示。

算法 3-1　Dijkstra 伪代码

```
初始化：起点站的所有站台节点 dist = 0，其余为 +∞，全部入堆
while 堆非空：
    d, node = pop_min()
    if node 已被扩展: continue
    if node.station == 终点站: 回溯前驱，结束
    for edge in adjacency[node]:
        nd = d + edge.time
        if nd < dist[edge.to] - ε:
            dist[edge.to] = nd
            prev[edge.to] = (node, edge.kind)
            heappush(nd, edge.to)
```

代码 3-1　松弛操作（core/algorithms.py 节选）

```
for edge in graph.adjacency[node]:
    nxt = (edge.to_station, edge.to_line)
    new_dist = current_dist + edge.time
    if new_dist < dist.get(nxt, math.inf) - 1e-12:
        dist[nxt] = new_dist
        prev[nxt] = (node, edge.kind)
        heapq.heappush(heap, (new_dist, next(counter), nxt))
```

起点的多源处理避免了为每条线路分别运行一次搜索；终点判定放在弹出站台节点时，只要弹出任意一个属于终点站的站台节点即可结束，结果仍然最优。若堆被清空仍未到达终点，则说明两站位于不同连通分量，返回 found=False。复杂度：设站台节点数为 V、有向边数为 E，每个站台节点最多入堆多次，总时间为 O((V+E) log V)，空间为 O(V+E)，用于存储邻接表、dist、prev 和堆。

3.2 A* 启发式搜索

A* 在 Dijkstra 的基础上为每个站台节点增加启发值 h(n)，按 f(n) = g(n) + h(n) 排序，其中 g(n) 是从起点到 n 的实际耗时，h(n) 是对 n 到终点剩余耗时的估计。本系统取 h(n) = 欧几里得距离(n, 终点) / v × 60，单位为分钟，v = 35 km/h 与所有乘车边使用的速度相同。

可采纳性：对任意乘车边 (u, v)，其实际耗时 c(u,v) = d(u,v)/v×60 + t_dwell ≥ d(u,v)/v×60。由三角不等式 d(u, t) ≤ d(u, v) + d(v, t)，可得 h(u) ≤ h(v) + c(u,v)；换乘边距离为 0 而耗时为 4 min，同样满足 h(u) ≤ h(v) + 4。因此 h 是一致的，一致的启发函数一定可采纳，A* 不会高估剩余代价，仍能得到与 Dijkstra 相同的最优解。

界面上选择“启发式最快（A*）”时，路径总耗时与 Dijkstra 完全相同，但扩展的站台节点明显减少：4.6 节的 5 对起终点中，A* 平均只扩展 76.8 个站台节点，而 Dijkstra 需要 133.0 个。这说明欧几里得距离能有效地把搜索引向终点方向。

代码 3-2 A* 启发函数与 f 值计算（core/algorithms.py 节选）

```
def heuristic(self, station_id: str, target_id: str) -> float:
    """直线距离 / 最高速度，换算成分钟（不会高估真实代价）。"""
    return self.distance(station_id, target_id) / self.speed_kmh * 60.0

f = new_g + graph.heuristic(edge.to_station, end_id)
heapq.heappush(heap, (f, new_g, next(counter), nxt))
```

3.3 BFS 最少换乘算法

最少换乘与最短耗时是两个不同的优化目标。系统把“换乘次数”作为层数：第 0 层是从起点出发可直接乘坐的线路，第 1 层是换乘一次可到达的站台节点，依此类推。每从队列中取出一个站台节点，就沿它所属的线路一次性扩展整条线——同一条线上的所有站点换乘次数都不变；这些站点上的其他线路站台节点换乘次数加一，放入下一层。终点第一次被访问时的层数就是最少换乘次数。

算法 3-2 分层 BFS 伪代码

```
队列初始化为起点站的所有站台节点，层号 level = 0
while 当前层非空 且 未到达终点:
    next_level = 空队列
    while 当前层非空:
        (station, line) = 出队
        for other in line_stations[line]:
            if other 未访问:
                记录 other 在第 level 层、父节点为 (station, line)
                if other == 终点: 找到，结束
                for other_line in station_lines[other]:
                    if other_line != line 且未访问:
```

```
                    入 next_level（换乘次数 +1）
  当前层 = next_level；level += 1
```

单纯按层扩展时，同一层内先被扩展到的线路可能绕远。为避免“同样换乘次数却选了一条耗时很长的路径”，系统在 BFS 求出最少换乘次数 T 之后，再把状态扩展为（站点, 线路, 已换乘次数），在换乘次数不超过 T 的约束下运行一次 Dijkstra，在保持最少换乘的前提下选择总耗时最短的路径。BFS 负责“换乘几次”，约束最短路负责“同换乘次数里怎么走最快”，这也是 4.6 节 BFS 时间不是最慢的原因之一。

复杂度：设线路数为 L、每条线路平均站数为 s，BFS 阶段每个站台节点最多入队一次，扩展线路时扫描其全部站点，第一阶段为 O(L·s + E)；约束最短路的状态数为 O(V·(T+1))，时间复杂度 O((V·T + E·T) log(V·T))。由于 T 通常不超过 3，本数据规模下开销仍然很小（实测中位数 0.33 ms）。

3.4 复杂度与适用场景对比

三种算法共享同一张站台图与同一套边权，区别只在优化目标与扩展方式。它们的复杂度与适用场景对比如表 3-1 所示。

表 3-1　三种算法复杂度与特点对比

算法	优化目标	时间复杂度	空间复杂度	最优性	适用场景
Dijkstra	总耗时最短	O((V+E) log V)	O(V+E)	最优	需要精确最快路径
A*	总耗时最短	最坏同 Dijkstra，通常更少	O(V+E)	最优（h 可采纳）	大地图、希望减少搜索量
BFS	换乘次数最少	O(L·s + (V·T+E·T) log(V·T))	O(V·(T+1))	换乘次数最优	少换乘、少步行出行

需要强调“最优性”的口径：Dijkstra 与 A* 优化的是总耗时，BFS 优化的是换乘次数，三者没有绝对的优劣。表 3-1 中 A* 的“最坏同 Dijkstra”是指启发函数失效时退化为 Dijkstra；实际效果取决于终点方向上的几何分布，4.6 节会给出实测数据。

3.5 异常处理

系统在数据加载、搜索和绘图三个环节都做了异常处理。数据错误由 loader 抛出带有字段或行号的 DataFormatError；两站不连通时算法返回 found=False，界面弹出“未找到路径”提示；起点与终点相同时直接返回零长度路径；绘图阶段对空数据、缺失线路、极小窗口做了防御性判断。异常与边界用例及预期行为如表 3-2 所示。

表 3-2　异常与边界用例

场景	输入/操作	系统行为
起点终点相同	西直门 → 西直门	返回零长度路径，时间与里程均为 0（自检 06）
两站不连通	苹果园 → 石厂（S1 支线）	返回 found=False，界面提示“未找到路径”（自检 07）
不连通合成图	disconnected_4.json：D1 → D4	返回 found=False，不绘制路径
非法文件类型	选择 .csv 数据文件	DataFormatError：仅支持 .json/.txt
JSON 语法错误	缺少逗号、坐标不是数字	DataFormatError 提示第几行第几列
线路引用不存在站点	lines 中写了未定义的站点 ID	加载失败并指出线路名与站点 ID
环线换乘方向	2 号线、10 号线绕行方向	按模运算取站数较少方向补齐中间站
满屏站名压盖	134 站、1400×900 窗口	按优先级省略站名，保证已画站名零重叠

3.6　系统实现与界面概览

系统分为四层：core/ 负责数据与算法（不 import Tkinter），ui/window.py 负责窗口、控件与状态机，ui/canvas.py 负责渲染与交互（不 import core、不读文件），selfcheck.py 与 tools/ 负责自检、导出、对比实验与报告生成。P3 的窗口只调用画布的公开方法，不直接 create_* 画布图元；P4 的画布通过归一化函数接受“像图/像结果”的对象，因此 P1/P2 改数据结构时画布无需跟着改。

P4 交付的 CanvasView 对外只暴露少量方法：bind_click(callback) 注册点击站点回调，show_path(result) 静态高亮并返回统计字典，animate(result, speed_level) 逐帧播放并返回同一份统计，reset() 清空路径与视图，export(path) 导出 PNG/PS。此外还提供 hit_test、set_zoom、pan_by、toggle_pause、skip_animation 等方法供按钮绑定。“不 import core”不是口头约定：自检第 10 项用 AST 扫描 ui/canvas.py，确认没有 core 导入、没有 open()、没有 solve()/load_dataset() 调用，防止以后把业务逻辑塞回画布。

主界面由标题栏、搜索工具栏、路网画布、结果面板与图例组成。工具栏提供起点/终点下拉框、三种算法选择、搜索、动画、暂停、跳过、复位、导出、动画速度（1×/2×/4×）、显示站名开关与“打开数据…”按钮；右侧面板显示总耗时、总里程、换乘次数、扩展节点数、途经站数与途经站点清单。加载 134 站数据、搜索“苹果园→天通苑北”的主界面如图 3-1 所示。



图 3-1　系统主界面（134 站数据，Dijkstra 搜索结果）

图 3-1 中画布按数据坐标等比例投影绘制线路与站点，换乘站用较大的双圈标记，站名采用碰撞检测自动避让，密集区域可滚轮放大查看；路径以"白色粗描边 + 玫红粗线"压在普通线路之上、站名之下，并在起点标"起"、终点标"终"、换乘站画橙色菱形。顶部信息卡汇总了 79.6 分钟、33.2 km、换乘 1 次、扩展 151 个站台节点、途经 32 站，与第 4 章的自检和对比实验数据完全对应。

第 4 章　正确性验证与结果分析

本章要解决的是“凭什么相信结果是正确的”：先给出运行环境与启动方式，演示界面功能，再说明渲染与交互的实现细节，然后覆盖异常与边界，接着用 10 项自检矩阵和手工推导两条独立证据链验证正确性，最后用 5 对起终点 × 3 算法的对比实验与性能数据说明效率差异。

4.1　运行环境与启动方式

系统使用 Python 3.10 及以上版本开发与验证（实测 Python 3.12.14），界面基于标准库 Tkinter（Tk 8.6），只有导出 PNG/JPG 与录制 GIF 需要第三方包 Pillow（实测 12.3.0，pip install pillow）；导出 PS 是矢量格式，不依赖 Pillow。运行环境为 Windows 11、1400×900 画布。最短启动与验证命令如下面的代码块所示。

代码 4-1　启动与验证命令

```
python main.py                          # 打开图形界面
python main.py --check                  # 10 项自检，退出码 0 = 全绿
python main.py --check --verbose        # 打印每一项的实测细节
python main.py --smoke _smoke/a.ps      # 界面冒烟：加载→搜索→动画→跳过→导出
python tools/bench.py                   # 5 对起终点 × 3 算法，写 docs/bench.csv
```

4.2　界面与功能演示

主界面如图 3-1 所示（对应 3.6 节）。典型操作流程是：在顶部下拉框选择起点“苹果园”与终点“天通苑北”，也可以直接在地图上左键点起点、右键点终点；选择算法后点“搜索”得到静态路径，点“动画”则以逐帧方式回放搜索与路径过程；动画过程中可以“暂停/继续”，也可以“跳过”直接落到最终结果。滚轮以光标为锚点在 0.5×–3× 之间缩放，按住左键拖拽平移，双击复位；点“导出图”可保存 PNG 或 PS。搜索完成后右侧面板同步显示耗时、里程、换乘次数、扩展节点数与途经站点清单。

以“苹果园→天通苑北”为例，Dijkstra 给出的最优路径为 79.6 分钟、33.2 km、换乘 1 次、途经 32 站，搜索过程扩展 151 个站台节点；切换为 A* 后结果数值完全相同，但扩展节点降为 115 个；切换为 BFS 后换乘次数仍为 1 次，但扩展计数口径不同（按线路扩展），只用于说明搜索规模小。同一 OD 下三种算法的差异在 4.6 节的表 4-3 中给出完整数据。

动画与立即显示的一致性由同一个 PathView 保证：show_path 与 animate 都先把 RouteResult 归一化成 PathView，所有统计值在动画开始前就已确定，动画只控制“画到第几个搜索事件、第几段路径”。因此暂停、继续、跳过都不会改变任何数值；自检第 10 项把两种方式的统计字典逐键比较，必须完全相等。

4.3 渲染与交互实现

渲染的核心是坐标变换。世界坐标是数据里的 x/y（单位 km，y 轴朝上），屏幕坐标是 Tkinter 画布像素（y 轴朝下），每次重绘都重新计算：screen_x = width/2 + (world_x − center_x) × scale + pan_x，screen_y = height/2 − (world_y − center_y) × scale + pan_y。x/y 用同一个 scale，窗口拉宽只增加留白、不会把城市压扁；y 前的减号就是 y 翻转。缩放时以光标为锚点，保证光标下的地点不动；窗口 <Configure> 事件做 120 ms 防抖，避免初始 1×1 尺寸时算错坐标。

路径绘制与标注分三层：先画线路与站点，再画路径（白色粗描边 + 玫红粗线），最后画站名与分段标注。站名避让采用“候选位置 + 64 像素网格索引 + 按优先级省略”的策略：每个站名尝试 8 个近位置与 6 圈远位置，只与附近网格内的矩形做碰撞检测；优先级依次为起点/终点 → 路径站 → 换乘站 → 普通站，仍放不下才省略。10 站演示图能画出 10/10 个站名与 5/5 段分段标注且零重叠，如图 4-1 所示。

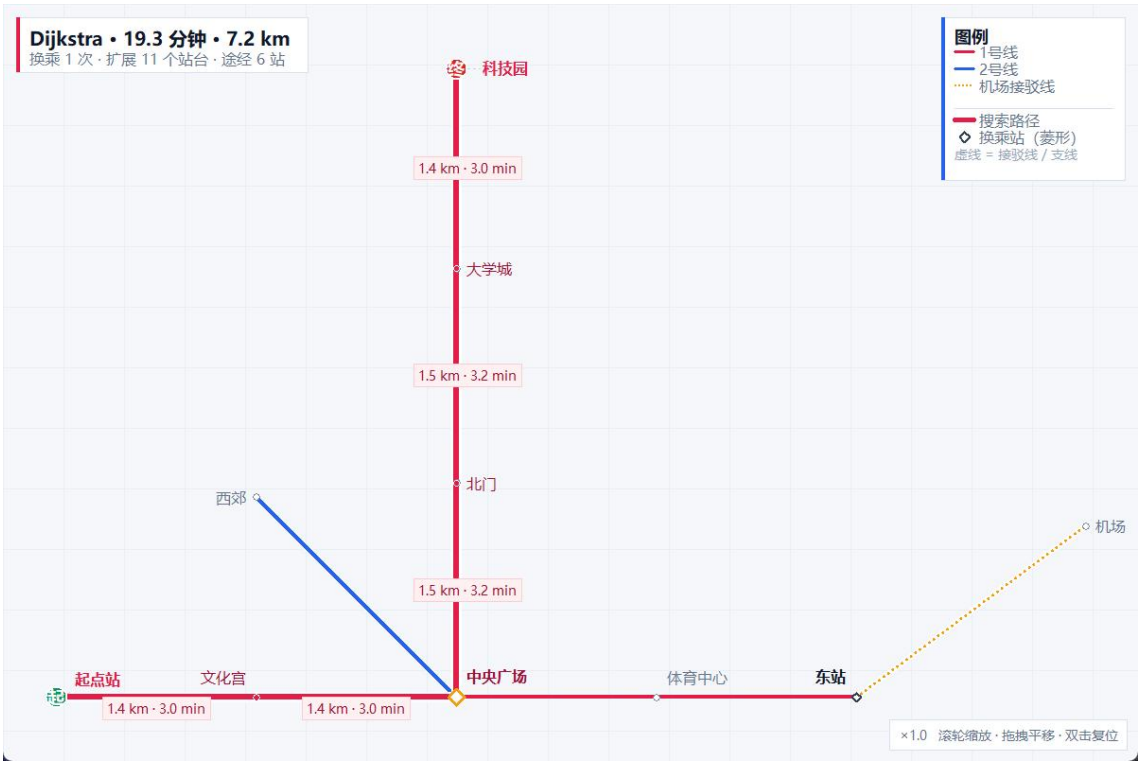


图 4-1　10 站演示图：10/10 站名与 5/5 分段标注（Dijkstra，19.3 min / 7.2 km / 换乘 1 次）

134 站的北京图在 1400×900 适配视图下无法同时显示全部站名，系统选择“保证已画出的站名零重叠、放不下的按优先级省略”，共画出 121 个站名、省略 13 个；路径的 31 个乘车分段中有 8 段标注可见、23 段因空间不足省略，放大视图后会重新排版并显示更多。北京路径的效果如图 4-2 所示。省略不是随机丢弃，而是按优先级依次尝试候选位置，因此重要站名（起点、终点、换乘站、路径站）优先保留。

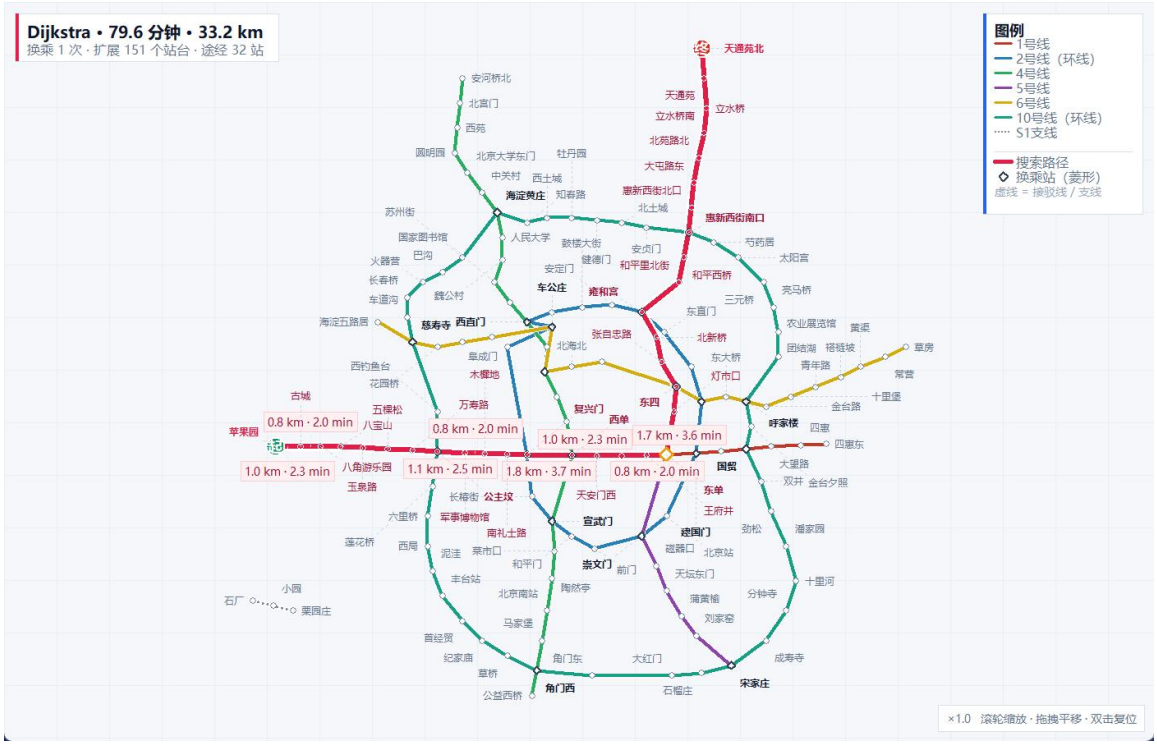


图 4-2　134 站北京图：121 个站名可见、13 个按优先级省略（苹果园→天通苑北，79.6 min）

缩放与命中是验收重点。画布允许 0.5×–3× 缩放，命中半径固定为 12 像素；自检第 10 项先把视图缩放 2.5 倍再平移，然后对 10 个站分别点在圆心与偏移 11 像素的位置，两次都必须命中同一个站，实测命中误差 11.0 像素（≤12 像素）。2× 缩放后的北京图如图 4-3 所示，放大后站名重新排版、之前省略的站名会重新出现，分段标注也显示得更多。

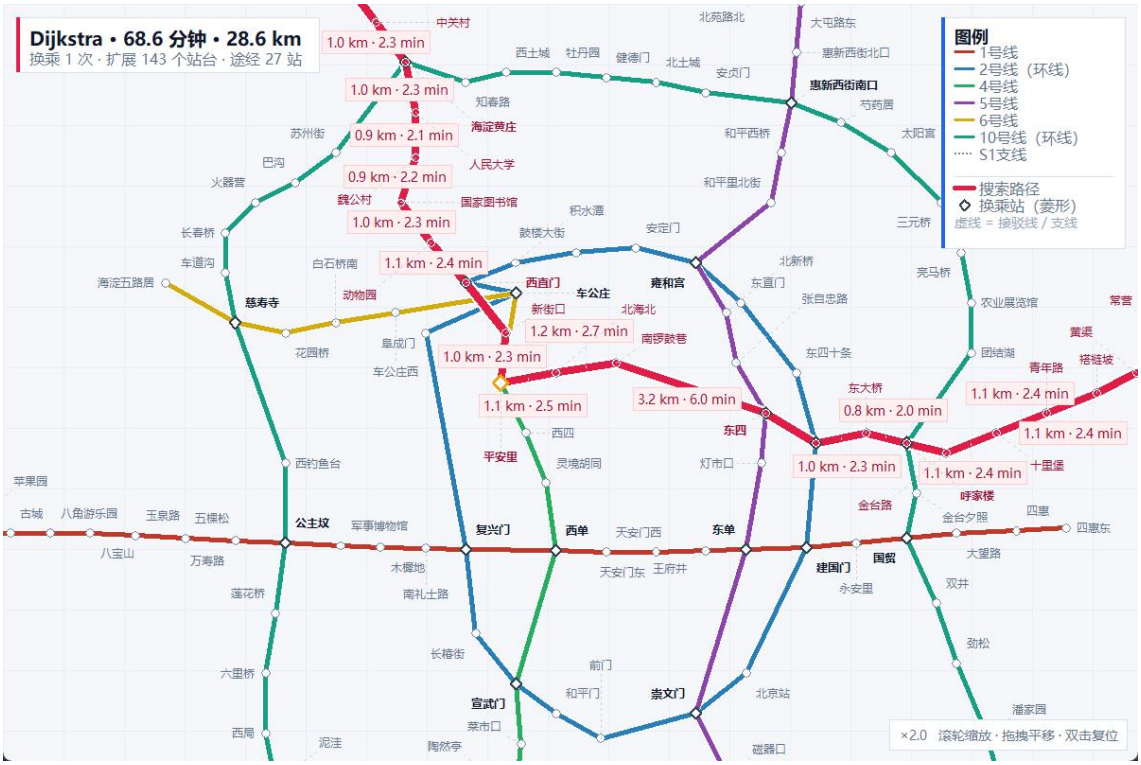


图 4-3　2× 缩放效果：站名重新排版，点击命中误差 11.0 像素（≤12 像素）

4.4　异常与边界测试

异常与边界由自检第 06、07 项覆盖，并在界面上有对应的提示行为。

• 起点终点相同：三种算法都直接返回零长度路径，总耗时与总里程均为 0，画布画一个“起终”徽标而不是一条路径（自检 06）。

• 不连通：合成图 disconnected_4.json 的 D1→D4 与真实数据中 S132（石厂，S1 支线）→S001（苹果园）都必须返回 found=False，界面显示红色“未找到路径”卡片，不画任何路径（自检 07）。

• 非法数据：非 .json/.txt 扩展名、JSON 语法错误、坐标不是数字、线路引用不存在的站点、同线站点重复、环线不足 3 站等，都会抛出 DataFormatError 并以中文对话框提示具体原因，程序不崩溃。

• 极端窗口与旧动画：窗口缩得很小时隐藏图例与信息卡、站名继续按空间省略；动画中切换算法或重新搜索会先 after_cancel 旧定时器并删除 anim 图层，自检断言 find_withtag("anim") == ()。

4.5 正确性验证

正确性不靠“看起来对”，而靠两条独立的证据链：第一条是 10 项自动自检矩阵（表 4-1），覆盖数据、算法、边界、性能与界面；第二条是手工推导（表 4-2），用三站图与 10 站演示图按公式自己算一遍时间、里程、换乘，再与程序输出对照。此外，自检第 03 项还会对 Dijkstra 的路径结果做独立重算：把路径拆成乘车段与换乘段，逐段检查“是否在线路上相邻”“分段里程/时间求和是否等于总里程/总时间”“换乘段数是否等于统计的换乘次数”，任何一项对不上就判红。

公式 4-1 乘车时间与换乘时间口径（与 P2 实现一致）

```
单段乘车时间（min） = 区间距离（km） / 列车速度（35 km/h） × 60 + 停站时间（0.6 min）
换乘时间（min）     = 4.0
总时间              = 所有乘车段 + 所有换乘段
```

自检矩阵的 10 项内容与失败含义如表 4-1 所示。它对应 main.py --check 的退出码闸门：10 项全绿退出码 0，任一项红退出码 1，可以直接作为合并前的自动关卡。

表 4-1　10 项自检矩阵

#	类别	检查内容	失败意味着
01	数据	北京 134 站 / 7 线、站名唯一、无孤立站	数据文件坏了
02	数据	10 站演示图、中央广场换乘、机场接驳线存在	自检素材不完整
03	算法	Dijkstra 路径端点/相邻性/分段求和的独立重算	搜索或结果封装错了
04	算法	A* 与 Dijkstra 同优且扩展节点更少	启发函数不可采纳
05	算法	BFS 换乘次数 ≤ Dijkstra	最少换乘逻辑错了
06	边界	起终点相同 → 0 分钟、1 站	边界处理崩溃
07	边界	不连通图/真实独立支线 → found=False	无路径处理错了
08	性能	建图 + 5 对起终点 × 3 算法耗时中位数	超过 200 ms 阈值
09	界面	等比、y 翻转、缩放锚点、0.5–3×、平移复位	坐标数学错了
10	界面	站名零重叠、动画与立即一致、拖拽、命中 ≤12 像素、导出、AST 扫描	渲染或交互回归

手工推导对照如表 4-2 所示。前三行来自 data/tiny_3.json（甲站、乙站、丙站三站一线，相邻两站均 1.2 km），最后一行是 data/demo_city.json 的 S1（起点站）→ S8（科技园）。前三行验证“最基础的一段/两段累加”没有算错，最后一行包含一次换乘，是完整公式的验证。

表 4-2　手工推导对照（误差 <0.001）

起终点	手算里程 /km	手算时间/min	程序里程 /km	程序时间 /min	误差
甲站 → 乙站	1.2	1.2/35×60+0.6 = 2.657	1.200	2.657	0
乙站 → 丙站	1.2	2.657	1.200	2.657	0

起终点	手算里程/km	手算时间/min	程序里程/km	程序时间/min	误差
甲站 → 丙站	2.4	2×2.657 = 5.314	2.400	5.314	0
起点站 → 科技园	7.2	3.000+3.000+4.000+3.1714+3.1714+3.000 = 19.342857 ≈ 19.343	7.200	19.343	<0.001

demo_city 的 S1→科技园 分段明细为：S1→S2、S2→S3 各 1.4 km/3.000 min，S3 换乘 4.000 min，S3→S6、S6→S7 各 1.5 km/3.171 min，S7→S8 为 1.4 km/3.000 min，合计 7.2 km、19.343 min、换乘 1 次、途经 6 站；程序输出的 distance_km=7.2、time_min=19.342857、transfers=1、station_count=6、expanded=11，与手算完全一致。

自检第 03 项还对 134 站真实数据做了一次独立重算：S001 苹果园 → S061 天通苑北 得到 32 站、79.6 min、33.2 km、换乘 1 次、扩展 151 个站台节点；逐段相邻性与求和误差均为 0。这说明“最优路径 + 结果封装”这一整条链路是正确的，而不只是三站小图上的巧合。

4.6 对比实验

对比实验的数据、起终点、算法与计时口径如下：数据为 data/beijing_metro.json（134 站 / 7 条线）；起终点共 5 对——苹果园→天通苑北、安河桥北→草房、四惠东→角门西、慈寿寺→崇文门、草房→安河桥北（含同线、跨线、环线、长距离）；算法为 Dijkstra（时间最短）、A*（时间最短 + 直线距离启发）与 BFS（最少换乘）；口径为每个组合热身 1 次、再计时 5 次取中位数，expanded 取 P2 结果里的扩展节点数。全部实验由 tools/bench.py 复现，原始数据见 docs/bench.csv，5 对起终点 × 3 算法的完整结果如表 4-3 所示。

表 4-3　5 对起终点 × 3 算法的对比实验结果（数据来自 docs/bench.csv）

起终点	算法	时间/min	里程/km	换乘次数	扩展节点数	耗时中位数/ms
苹果园→天通苑北	Dijkstra	79.6	33.2	1	151	0.32
苹果园→天通苑北	A*	79.6	33.2	1	115	0.34
苹果园→天通苑北	BFS	79.6	33.2	1	5	0.37
安河桥北→草房	Dijkstra	68.6	28.6	1	143	0.29
安河桥北→草房	A*	68.6	28.6	1	78	0.24
安河桥北→草房	BFS	68.6	28.6	1	4	0.33
四惠东→角门西	Dijkstra	46.2	19.0	1	112	0.22
四惠东→角门西	A*	46.2	19.0	1	48	0.16
四惠东→角门西	BFS	46.2	19.0	1	2	0.24
慈寿寺→崇文门	Dijkstra	40.6	15.1	2	114	0.22
慈寿寺→崇文门	A*	40.6	15.1	2	57	0.17
慈寿寺→崇文门	BFS	43.2	19.0	1	3	0.37

起终点	算法	时间/min	里程/km	换乘次数	扩展节点数	耗时中位数/ms
草房→安河桥北	Dijkstra	68.6	28.6	1	145	0.30
草房→安河桥北	A*	68.6	28.6	1	86	0.28
草房→安河桥北	BFS	68.6	28.6	1	4	0.33

注：扩展节点均值为 Dijkstra 133.0、A* 76.8、BFS 3.6；单次寻路耗时中位数均值为 Dijkstra 0.27 ms、A* 0.24 ms、BFS 0.33 ms。BFS 按整条线路扩展，与 Dijkstra/A* 的逐站台扩展不是同一计数口径。

表 4-3 对应的扩展节点数对比可以更直观地看出 A* 的剪枝效果，如图 4-4 所示。图中柱高为扩展节点数，BFS 的柱子接近 0 只是因为计数口径不同，主要应比较 A* 相对 Dijkstra 的下降。

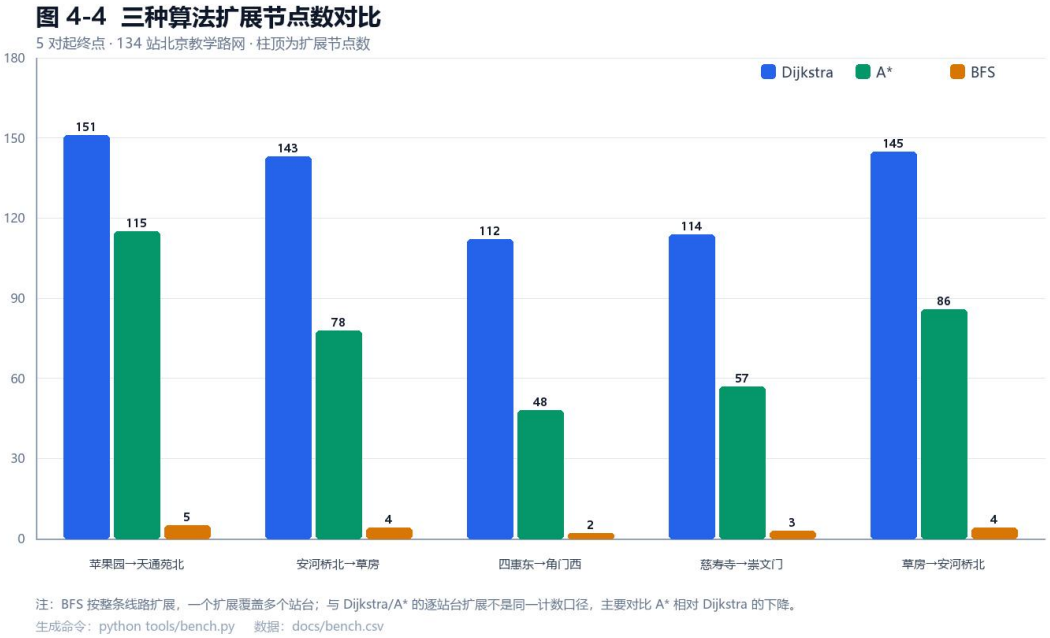


图 4-4　三种算法在 5 对起终点上的扩展节点数对比（BFS 为按线路扩展口径）

由表 4-3 与图 4-4 可以归纳出三条结论。

（1）A* 与 Dijkstra 同优。五对起终点的最短时间完全一致（差值小于 1e-6），但 A* 平均只扩展 76.8 个站台节点，比 Dijkstra 的 133.0 个少 42.3%；单对最多从 112 降到 48（少 57.1%，四惠东→角门西），最少从 151 降到 115（少 23.8%，苹果园→天通苑北）。这说明直线距离启发函数既保持了最优性，又显著缩小了搜索空间。

（2）BFS 确实最少换乘。前三对起终点 Dijkstra 与 BFS 的换乘次数相同，第四对“慈寿寺→崇文门”体现了准则差异：Dijkstra 用 2 次换乘换到 40.6 min，BFS 只用 1 次换乘但需要 43.2 min，多花 2.6 min（里程从 15.1 km 增加到 19.0 km）。这就是“最快”与“最少换乘”两种准则的真实取舍，也是三算法对比最值得写的一段。

（3）BFS 的扩展数不能直接和另外两个比。BFS 一次扩展整条线路，一个“扩展”覆盖多个站台节点；Dijkstra 与 A* 是逐站台扩展。因此表 4-3 和图 4-4 里 BFS 的 2–5 个扩展只说明它搜索规模小，不能与 133.0 直接比较；真正同口径的对比是 A* 相对 Dijkstra 的下降幅度。

4.7 性能与观感

性能口径分两类：算法耗时是 P2 内部用 perf_counter() 测得的毫秒数，不含画布绘制；画布耗时是在 1400×900、134 站、带路径的条件下实测的。五对起终点的耗时中位数均值分别为 Dijkstra 0.27 ms、A* 0.24 ms、BFS 0.33 ms，都远低于 200 ms 的验收阈值；画布首次绘制约 200 ms（含首次字体测量），之后整图重绘约 80 ms，拖拽时只平移已画图元、不重排文字，每帧仅数毫秒；导出 PNG 约 3 s（含等待窗口合成 0.35 s），PS 为矢量、几乎瞬间完成；动画单帧 tick 小于 10 ms，只删除/重画 anim 图层。

渲染性能主要靠三处优化：线路用一条折线画完，而不是每个区间画一条；134 个站名的文字宽度只测量一次并缓存，站名排版用 64 像素网格索引，只检查附近矩形；拖动与滚轮缩放时把 16 ms 内的重绘请求合并，窗口 <Configure> 再做 120 ms 防抖。这几处优化把 134 站图的重绘从数百毫秒压到约 80 ms，界面在缩放与拖拽时保持跟手。

观感上还有两点取舍。第一，密集路网无法既“全部显示站名”又“零重叠”，系统选择保证零重叠、按优先级省略，并在图例与状态栏提示省略数量；放大后之前省略的站名会重新出现，这比把所有站名挤在一起更符合验收要求。第二，分段标注（km/min）先于站名排版、尽量靠近路径，10 站演示图 5 段全部显示，134 站全城图放不下的标注自动省略，放大后会显示更多。

动画方面，1×/2×/4× 通过定时器间隔换算（30 ms、80 ms 两档除以速度），不使用 sleep 阻塞界面；暂停 = 取消定时器且不排下一帧，继续 = 再排一帧，跳过 = 直接落到最终静态图；切换算法或重新搜索时先 after_cancel 旧任务并清空 anim 图层，自检断言画布上不再残留 anim 图元。

第 5 章　总结与收获

本章要解决的是“这门课学到了什么”：先回到数据结构本身，说明图、堆、队列在项目中的真实作用，再比较三种算法在效率、公平与可解释性上的差异，然后把开发中踩过的坑按“现象→原因→解法”整理出来，最后给出局限与改进方向。

5.1　图、堆、队列在本项目中的实际作用

图是本项目的骨架。系统把地铁网络建成“站台节点图”——站台节点是（站点, 线路），边分乘车边与换乘边。这样换乘次数、换乘时间都能精确建模，而不是把换乘藏成一个普通站点；路径中每经过一条换乘边，换乘次数加一，统计与展示都自然成立。

堆是 Dijkstra 与 A* 的心脏。两者都用 heapq 维护待扩展的站台节点，每次取当前代价最小者；134 站的稀疏图用邻接表存储，空间 O(V+E)，比邻接矩阵省一个数量级。堆里额外放一个自增计数器，解决了距离相同时元素不可比较的问题，也让实验可复现。

队列是 BFS 的骨架。分层 BFS 用 deque 按层扩展，每层是一批“再换乘一次可达”的站台节点，因此终点第一次被访问时的层数就是最少换乘次数。渲染里还用到一种“空间索引”思想：站名避让用 64 像素网格，只检查附近矩形，把 134 个站名的两两比较降为常数级，这与哈希表“用空间换时间”的思路一脉相承。

5.2　三算法在效率、公平与可解释性上的差异

效率上，Dijkstra 扩展最多但最稳；A* 用直线距离做启发，搜索规模平均少 42.3%，且因为启发函数可采纳，结果仍是最优；BFS 按线路扩展，扩展计数最少，但每次扩展要扫一条线，实际耗时不一定最低（实测中位数 0.33 ms，略高于 Dijkstra 的 0.27 ms）。

公平上，Dijkstra/A* 优化“总时间”，BFS 优化“换乘次数”，两者没有绝对优劣。慈寿寺→崇文门的例子说明少换乘可能多花 2.6 分钟；报告里不能简单说“BFS 更差”，而要说“准则不同、用户按偏好选择”。

可解释性上，Dijkstra 的“按代价从小到大扩展”最好讲；A* 需要额外解释启发函数不高估剩余代价；BFS 的“换乘层数”最直观，适合给非技术用户解释“为什么要这

样换乘”。三者放在一起，正好对应“最优但慢热”“最优且更快”“少换乘但可能更久”三种真实需求。

5.3 开发中的问题与解法

开发过程中踩过的坑按“现象→原因→解法”整理为以下 7 条，覆盖渲染、动画、算法与工程化四个方面。

（1）首次绘制时坐标全错。现象：窗口刚打开时地图缩成一点或跑到屏幕外。原因：Tk 在窗口还没真正显示时 winfo_width() 返回 1，按 1×1 算出的比例必然错误。解法：首次 <Configure> 后做 120 ms 防抖重绘，等尺寸稳定再计算；同时保留“每次重绘都从世界坐标重算”的约定，不缓存屏幕坐标。

（2）固定像素画图导致缩放糊、变形。现象：放大后线条变粗、站名错位。原因：把屏幕像素当成坐标保存下来，缩放时只做整体拉伸。解法：视图状态只保存 zoom、pan 与视口尺寸，每次重绘都从世界坐标重新投影，并且 x/y 使用同一个 scale；缩放以光标为锚点，保证光标下的地点不动。

（3）动画 sleep 导致界面假死。现象：播放动画时窗口拖不动、按钮点不动。原因：在主线程里 sleep 等下一帧。解法：改用 after() 链式调度；暂停 = 取消定时器且不排下一帧，继续 = 再排一帧；切换算法先 after_cancel 旧任务并清空 anim 图层；跳过 = 直接落到最终结果。

（4）环线路径重建把整条线当成一段。现象：2 号线、10 号线的换乘路径出现异常大的耗时（上千分钟）。原因：按站点下标区间截取中间站时，环线的首尾相邻边（如积水潭→西直门）被当成跨越整条线。解法：环线用模运算分别算顺时针、逆时针步数，取站数较少的方向补齐中间站；自检第 03 项的“路径相邻性”复核专门防这个问题回归。

（5）BFS 的换乘自环。现象：BFS 长时间不结束，前驱链出现环。原因：扩展线路时把“当前线路本身”也放进了换乘候选，产生（站台节点, 当前线路）→自身的边，层次多算一次。解法：跳过与当前线路相同的线路，并把已乘过的站台节点标记为已访问；同时用 visited_platforms 去重，保证每个站台节点只入队一次。

（6）134 站站名互相压盖。现象：全城视图下站名糊成一片，看不清是哪一站。原因：直接按站点坐标绘制全部站名，没有碰撞检测。解法：候选位置 + 64 像素网格

索引 + 按优先级（起点/终点 → 路径站 → 换乘站 → 普通站）依次尝试 8 个近位置和 6 圈远位置，放不下才省略，从而保证“已画出的站名零重叠”。

（7）高分屏导出截偏。现象：PNG 截图偏掉一块或截到桌面。原因：Tk 使用逻辑像素，而截图按物理像素抓取，Windows 125%/150% 缩放下两者不一致；窗口合成（DWM）也比 Tk 的 update() 慢半拍。解法：创建 Tk 之前声明 DPI 感知（ui/dpi.py），截图前泵 0.35 s 事件循环等窗口真正上屏，再按 winfo_rootx/rooty 抓取。

5.4 局限与改进

当前系统仍有明确的局限：数据是教学示意坐标，不是真实地理投影；换乘时间固定 4 分钟，没有考虑通道长短与站内步行距离；列车匀速、各线路速度相同，也没有高峰/平峰与拥挤度差异。这些假设让实验可复现，但与真实出行仍有距离。

- 标注排版：密集图仍会省略部分站名与分段标注，后续可做全局标签排版（如力导向、模拟退火），或把标注改成“悬停显示”。
- 输出格式：目前是静态 PNG/PS 快照，后续可支持导出 SVG、批量生成多组对比图。
- 多准则：算法只支持“时间最短”和“换乘最少”两种准则，后续可加入“步行最少”“票价最低”等多权重，甚至做 Pareto 前沿。
- 算法可视化：当前动画是“搜索结果回放”，不是真正的算法过程可视化；如果要展示堆操作，需要在 P2 里输出更细的事件类型（如入堆、出堆、松弛成功）。
- 规模化：可尝试双向 Dijkstra、Contraction Hierarchies 等加速方法，并把界面迁移到 Web 端，以支持更大规模路网和移动设备访问。

参考文献

[1] 严蔚敏, 吴伟民. 数据结构（C 语言版）[M]. 北京: 清华大学出版社, 2007.

[2] Cormen T H, Leiserson C E, Rivest R L, et al. Introduction to Algorithms[M]. 3rd ed. Cambridge: MIT Press, 2009.

[3] Dijkstra E W. A note on two problems in connexion with graphs[J]. Numerische Mathematik, 1959, 1(1): 269-271.

[4] Hart P E, Nilsson N J, Raphael B. A formal basis for the heuristic determination of minimum cost paths[J]. IEEE Transactions on Systems Science and Cybernetics, 1968, 4(2): 100-107.

[5] Python Software Foundation. Python 3 Documentation[EB/OL]. https://docs.python.org/3/, 2026.

[6] Python Software Foundation. tkinter — Python interface to Tcl/Tk[EB/OL]. https://docs.python.org/3/library/tkinter.html, 2026.

[7] Pillow Contributors. Pillow (PIL Fork) Documentation[EB/OL]. https://pillow.readthedocs.io/en/stable/, 2026.

附录 A　复现命令

以下命令均在仓库根目录 metro-p4/ 下执行，覆盖自检、导出、对比实验、录屏与报告生成的全过程。

代码 A-1　全部复现命令

```
pip install pillow                              # 只有 PNG/GIF 导出需要
python main.py --check                          # 10 项自检，退出码 0
python main.py --check --verbose                # 打印每项实测细节
python main.py --smoke _smoke/a.ps              # 界面冒烟 + 导出画布
python tools/export_canvas.py --figure-pack     # 重新生成报告插图
python tools/bench.py                           # 5 对 OD × 3 算法，写 bench.csv
python tools/record_demo.py                     # 自动录演示 GIF
python selfcheck.py --mutation-demo             # 3 个 CAUGHT，证明断言有判别力
python tools/make_report_assets.py              # 生成图 2-1 并校验插图
python tools/make_report.py                     # 生成报告终稿 docx
python tools/check_report.py                    # 统稿检查，退出码 0
```

其中 python tools/make_report.py 会重新计算表 4-2、表 4-3 与摘要中的数字（直接读 docs/bench.csv 与真实数据），python tools/check_report.py 会校验五章结构、图表编号、术语、数字一致性与插图嵌入情况，两者都把退出码作为闸门。

附录 B　文件清单

最终仓库的文件组织如表 B-1 所示。

表 B-1　项目文件清单

文件/目录	说明
main.py	程序入口：GUI / --check 自检 / --smoke 界面冒烟
selfcheck.py	10 项自检矩阵 + mutation 判别力演示
core/graph.py	Station、MetroLine、Edge 与 MetroGraph（站台节点建图）
core/loader.py	JSON/TXT 解析、统一校验、DataFormatError
core/algorithms.py	Dijkstra、A*、分层 BFS 与 RouteResult 封装
ui/window.py	Tkinter 窗口、控件、状态机与统计面板
ui/canvas.py	CanvasView：渲染、交互、动画、导出（P4 核心）
ui/dpi.py	Windows 高 DPI 适配（保证截图不偏）
tools/export_canvas.py	画布导出 PNG/PS，--figure-pack 一键生成插图
tools/bench.py	5 对起终点 × 3 算法对比实验，生成 bench.csv 与图 4-4
tools/record_demo.py	自动驱动真实窗口录制演示 GIF
tools/report_common.py	Word 排版工具：封面/摘要/目录域/标题/图表
tools/make_report_assets.py	绘制图 2-1 架构图并校验全部插图
tools/make_report.py	生成本报告 docx（读真实数据与 bench.csv）
tools/check_report.py	统稿检查：结构、编号、术语、数字、插图
data/	北京 134 站教学路网 + 10 站演示图 + 三站/不连通小图
docs/	验收说明、报告草稿、bench.csv、插图与演示 GIF
reports/	报告终稿 docx（由 tools/make_report.py 生成）
README.md	项目说明、5 分钟运行指南、接口与 FAQ

附录 C　自检输出节选

以下为 python main.py --check 在本机的真实输出节选（Windows 11、Python 3.12.14、Tk 8.6、Pillow 12.3.0），10 项全部通过、退出码 0。

代码 C-1　自检输出节选

```
城市交通路径导航系统 · P4 自检矩阵
Python 3.12.14 · 项目目录 …/metro-p4
----------------------------------------------------------------
[PASS] 01. 北京路网数据完整  (数据, 6 ms)
        134 站 / 7 条线 / x∈[-14.6,11.6] y∈[-9.8,16.2]
[PASS] 02. 10 站演示图与接驳线  (数据, 1 ms)
        10 站 / 3 线 / 换乘站 2 个
[PASS] 03. Dijkstra 路径自洽（独立重算）  (算法, 2 ms)
        32 站 / 79.6 min / 33.2 km / 换乘 1 / 扩展 151
[PASS] 04. A* 与 Dijkstra 同优且扩展更少  (算法, 3 ms)
        同优 68.6 min；扩展 A*=78 / Dijkstra=143（省 65 个）
[PASS] 05. BFS 换乘次数最少  (算法, 4 ms)
        BFS 换乘 1 次 / 79.6 min；Dijkstra 换乘 1 次 / 79.6 min
[PASS] 06. 边界：起终点相同  (边界, 2 ms)
[PASS] 07. 边界：不连通返回无路径  (边界, 2 ms)
        合成图 D1→D4 与真实图 S132→S001 均正确返回无路径
[PASS] 08. 134 站全图性能  (性能, 16 ms)
        建图 2.1ms；5 对起终点 × 3 算法，最慢中位数 0.40ms
[PASS] 09. 坐标变换：等比 + y 翻转 + 缩放锚点  (界面, 0 ms)
        等比 90.000px/km；缩放范围 0.5×–3.0×；往返误差 <1e-6 px
[PASS] 10. 界面：站名零重叠 + 动画一致 + 命中 + 导出  (界面, 2147 ms)
        AST：无 core 导入、无 open()/solve()/load_dataset()；10 站名零重叠；
        5 段标注零重叠；动画/立即一致；命中误差 11.0px；PNG 39KB
----------------------------------------------------------------
结论：10 通过 / 0 失败 / 0 跳过，共 10 项，用时 2184 ms
闸门：绿（退出码 0）。
```